Ders 23 Genişletilmiş Notu: Geri İzleme: Alt Küme Toplamı ve N-Vezir Problemi; Dal ve Sınır ile Atama Problemi

Ders 23 Genişletilmiş Notu: Geri İzleme: Alt Küme Toplamı ve N-Vezir Problemi; Dal ve Sınır ile Atama Problemi

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Geri İzleme Fikri
- 2. Durum Uzayı Ağacı: Umut Veren ve Umut Vermeyen Düğümler
- 3. Alt Küme Toplamı Problemi
- 4. N-Vezir Problemi
- 5. Bilgisayarda N-Vezir: Çapraz Kontrol ve Dizi Gösterimi
- 6. Genel Geri İzleme Algoritması
- 7. Arama Uzayını Küçültme Hileleri
- 8. Geri İzlemenin Karmaşıklığı
- 9. Dal ve Sınır: Optimizasyon Problemleri
- 10. Atama Problemi ve En İyi Öncelikli Dal ve Sınır
- Örnek Sorular

1. Geri İzleme Fikri

Dönemin 12. haftasında, son üç haftaya girilirken hoca bu haftanın konularını geri izleme (backtracking) ve dal ve sınır (branch and bound) olarak açıkladı. Ders Zoom üzerinden yapıldı. Hoca konuları tablet üzerinde el yazısıyla anlattı.

Geri izlemenin temel fikri şudur: Belirli bir hedefe ulaşmaya çalışırken geçtiğimiz yolları kaydederiz ve her durumdan (state) bir sonrakine ilerleyerek çözüme varmaya çalışırız. Bu yollar üzerinde çıkmaz noktalar (dead end) vardır, yani oradan hedefe ulaşılamaz. Böyle bir noktaya geldiğimizde bir önceki duruma geri döneriz. O durumdan başka bir yol varsa onu deneriz. Yoksa bir adım daha geri gideriz. Hoca bunu ekranda bir ağaç üzerinde gösterdi: Bir dalın sonunda çıkmaz sokak, geri dönüş, başka bir dal, yine çıkmaz sokak, iki adım geri dönüş ve en sonunda çözüme (solution) giden bir yol.

Hocaya göre bu yaklaşıma verilebilecek en güzel örnek labirenttir. Labirentin tek bir çıkışı (exit) olsun. İlerlerken bir duvarla karşılaştığımızda bu duvar bize daha fazla ilerleyemeyeceğimizi söyler ve geri döneriz. Labirentteki temel prensip şudur: Bir yolu dene; çıkmaz sokaksa geri dön; başka seçenek kalmadıysa bir adım daha geri dön; bu şekilde çıkışa kadar ulaş. Hoca labirentin içinde bütün seçenekleri tek tek deneyen bir fare düşünmemizi istedi.

Geri izleme aslında derinlik öncelikli arama (depth first search, DFS) mantığını kullanır, üstüne de budama (pruning) yapar. Budama, ağacın belirli bir noktasındaki seçeneklerin ortadan kaldırılmasıdır. Bir kesme işlemi yapılır ve aramaya ağacın diğer kısmından devam edilir.

Bu yaklaşımın iyileştirmeye çalıştığı yöntem kapsamlı aramadır (exhaustive search). Kombinatoryal problemlerde basit, kaba kuvvet (brute force) bir yaklaşımla bütün olasılıkları denemek

gerekir. Eleman sayısı arttıkça bu sayı 2^n ya da $n!$ mertebesine çıkar. n yüzlere çıktığında polinom zamanda çözülemeyen problemler ortaya çıkar. Geri izleme, çözüme götürme olasılığı olmayan seçenekleri devre dışı bırakarak arama uzayını (search space) daraltır. Hoca, dersin ikinci bölümünde görülecek dal ve sınır algoritmasının da aynı yaklaşımla ilerlediğini, yalnızca optimizasyon problemlerine yöneldiğini söyledi.

2. Durum Uzayı Ağacı: Umut Veren ve Umut Vermeyen Düğümler

Geri izlemede her adımı gösteren bir ağaç kullanılır. Bu ağaca durum uzayı ağacı (state space tree) denir. Ağaçtaki düğümler için iki durum söz konusudur:

- **Umut veren düğüm (promising node):** Geldiğimiz noktada hâlâ bakılabilecek bir ihtimal, bir seçenek vardır. Labirentte bir adım sonra henüz duvara çarpmadıysak o nokta umut vericidir.
- **Umut vermeyen düğüm (non-promising node):** O noktadan sonra çözüme götürecek bir ihtimal kalmamıştır.

Hoca bir ayrıntının altını çizdi: Labirent örneğinde umut vermeyen düğümün altında zaten başka seçenek yokmuş gibi görünür. Fakat sonraki problemlerde umut vermeyen bir düğümün altına eklenebilecek seçenekler hâlâ olacaktır. O seçenekler artık ihtimal dahilinde olmadığı için onları kontrol etme gereği duymayız. Böylece, şanslıysak, arama uzayımızı oldukça daraltmış oluruz.

3. Alt Küme Toplamı Problemi

3.1 Problemin tanımı

Alt küme toplamı problemi (sum of subsets) şudur: $A = \{a_1, a_2, \ldots, a_n\}$ kümesinde n tane pozitif tam sayı var. Bunlardan hangilerini toplarsak verilen bir d tam sayısını elde ederiz?

Hocanın ilk örneği: $A = \{1, 2, 5, 6, 8\}$ ve $d = 9$. Gözle bakınca iki seçenek çıkar: $\{1, 2, 6\}$ ve $\{1, 8\}$. Bazen yalnızca bir çözümün peşinde olabiliriz, ilkini bulunca sonuca ulaşmış oluruz. Bazen de hepsi istenir. Kaba kuvvetle bu iki seçeneğe ulaşmak için, $n = 5$ olduğundan, $2^5 - 1 = 31$ farklı kombinasyonu denemek gerekir. $n = 1000$ olduğunda bu sayı $2^{1000} - 1$'e çıkar ve çok daha fazla işlem gücü gerektirir. Geri izlemeyle amaç bu işlem sayısını azaltmaktır.

Python ile kontrol edildi: Bu kümede toplamı 9 olan alt kümeler yalnızca $\{1, 8\}$ ve $\{1, 2, 6\}$'dır.

3.2 Neden önce sıralama?

Hocanın ikinci örneği: $S = \{3, 5, 6, 7\}$ ve $d = 15$. Hoca, sayılar karışık verilseydi ilk yapılacak işin onları küçükten büyüğe sıralamak olacağını söyledi. Bu örnekte sayılar zaten sıralı verildi. Sıralamanın sebebi az sonra görülecek iki budama koşulunun ancak sıralı dizide işe yaramasıdır.

3.3 Durum uzayı ağacının kurulması

Başlangıçta elimizde hiç sayı yoktur ve toplam 0'dır. Her seviyede sıradaki sayı için iki seçenek vardır: o sayıyı toplama eklemek ya da eklememek. Hedefin 15 olduğunu akılda tutarak ağaç şöyle kuruldu:

- **3 alındı (3), 5 alındı (8), 6 alındı (14):** $14+7 = 21 > 15$. Bu düğüm umut vermez. Hoca, 7'den sonra kümede 9, 12, 28 gibi sayılar olsaydı bile bunların hiçbirine bakmayacağımızı vurguladı. Sayılar sıralı olduğu için onlar da 15'i aşardı.
- **3 alındı, 5 alındı, 6 alınmadı (8):** 7 alınırsa $8 + 7 = 15$. **Çözüm.** 7 alınmazsa toplam 8'de kalır ve bu bir ihtimal olmaktan çıkar.
- **3 alındı, 5 alınmadı, 6 alındı (9):** $9 + 7 = 16 > 15$. Umut vermez.
- **3 alındı, 5 alınmadı, 6 alınmadı (3):** $3 + 7 = 10 < 15$. Umut vermez.
- **3 alınmadı, 5 alındı, 6 alındı (11):** $11 + 7 = 18 > 15$. Umut vermez.
- **3 alınmadı, 5 alındı, 6 alınmadı (5):** $5 + 7 = 12 < 15$. Umut vermez.

- **3 alınmadı, 5 alınmadı (0):** Geriye kalan sayılar 6 ve 7'dir, toplamları 13'tür. 6 ve 7 eklenmeden bile 15'e ulaşılamayacağı görülür. Bu dal daha 6 ve 7'ye bakılmadan seçenek dışı kalır.

Sonuç olarak 15'e ulaşmak için 3, 5 ve 7 kullanıldı. Ağaç üzerinde ilerleyerek hangi sayıların bu toplamı oluşturduğunu bulmuş olduk.

Python ile kontrol edildi: Aynı budama kurallarıyla yazılan geri izleme, ağaçta tam olarak yukarıdaki altı umut vermeyen düğümü ve tek çözüm olarak $\{3, 5, 7\}$'yi buldu.

3.4 Budama koşulları

Hoca ardından koşulları genel olarak tahtaya İngilizce yazdı. s o ana kadarki toplam, i son karar verilen sayının indisi olsun. $s \neq d$ ise, aşağıdaki iki eşitsizlikten biri sağlandığında düğüm umut vermez olarak işaretlenir ve o daldan aşağı inilmez:

- $s + a_{i+1} > d$: Toplam çok büyük (sum is too large). Sıradaki en küçük sayıyı eklemek bile hedefi aşıyor. Sayılar sıralı olduğu için sonrakiler de aşar.
- $s + \sum_{j=i+1}^{n} a_j < d$: Toplam çok küçük (sum is too small). Geriye kalan bütün sayıları eklesek bile hedefe yetişemiyoruz.

3.5 Geri izleme bir garanti vermez

Hoca bu noktada "eğer şansımız da yardım ediyorsa" ifadesini açıkladı. Bu çözümün her girdi ve her problem için az sayıda işlemle biteceğinin garantisi yoktur. Girdiye bağlı olarak çok sayıda durumu denemek zorunda kalabilirsiniz. Bu tür algoritmalar size belirli bir polinom zamanda en iyi sonucu yüzde yüz garanti etmez. Fakat kapsamlı aramayla karşılaştırıldığında, istenen zamanda sonuca ulaşma şansınız çok daha yüksektir.

4. N-Vezir Problemi

4.1 Problem ve kaba kuvvet seçenekleri

N-vezir (N-queens) problemi literatürde sıklıkla anlatılan bir problemdir: Vezirler satranç tahtasına birbirlerini yemeyecek şekilde yerleştirilir. Vezir yatayda, dikeyde ve çaprazda hareket edebilir.

Hoca 8-vezir problemi için, kullandığımız bilgi arttıkça kontrol edilmesi gereken seçenek sayısının nasıl azaldığını gösterdi:

- **Her vezir herhangi bir karede:** Tahtada 64 kare, 8 vezir var. Olasılık sayısı $\binom{64}{8} = \frac{64!}{8!\cdot 56!} = 4\,426\,165\,368$. İlk akla gelen çözüm olduğu için buna kaba kuvvet çözümü denir.
- **Her satırda bir vezir:** Aynı satırdaki iki vezir birbirini yer. Her vezir ayrı bir satıra konursa olasılık $n^n = 8^8 = 16\,777\,216$'ya düşer.
- **Her satır ve her sütunda bir vezir:** Olasılık $n! = 8! = 40\,320$'ye iner.
- **Geri izleme:** Yaklaşık 2057 adımda vezirlerin konumuna karar verilir. Hoca bunun $8!$'e göre yaklaşık 20 kat daha az işlem olduğunu ve oldukça tatminkâr bir sonuç olduğunu söyledi.

Python ile kontrol edildi: $\binom{64}{8}$, 8^8 ve $8!$ değerleri doğru. $40\,320/2057 \approx 19{,}6$, yani "yaklaşık 20 kat" doğru. Basit bir geri izlemenin 8-vezirin bütün 92 çözümünü bulurken ziyaret ettiği düğüm sayısı, kök (boş tahta) dahil, tam 2057'dir. İlk çözüm 114. düğümde bulunur.

4.2 Küçük tahtalar

8-vezir ağacını burada modellemek zaman alacağı için hoca 4-vezir üzerinden ilerledi. Önce küçük tahtalara baktı ve bunların derste sorulabilecek, çok zor olmayan sorular olduğunu, ama bu kez sormadan doğrudan anlatacağını söyledi:

- 2×2 tahtada vezirleri hangi şekilde koysanız birbirlerini yer. Çözüm yoktur.
- 3×3 tahtada da vezirleri birbirini yemeyecek şekilde yerleştiremezsiniz.
- En az 4×4'lük bir tahta gerekir. Dört vezir birbirini yemeyecek şekilde yerleştirilebilir.

Ders sonrası not: Hoca 1×1 tahtada, yani 1-vezir probleminde, çözüm olmadığını söyledi; doğrusu bir çözüm vardır: tek vezir tek kareye konur ve tehdit edeceği başka vezir yoktur. Çözümü olmayan durumlar $n = 2$ ve $n = 3$'tür. Python ile kontrol edildi: $n = 1$ için 1, $n = 2$ ve $n = 3$ için 0, $n = 4$ için 2 çözüm vardır. Bu yüzden $n = 1$ önemsiz (trivial) bir durumdur ve en küçük anlamlı örnek $n = 4$'tür.

4.3 4-vezir için durum uzayı ağacı

Satır ve sütunlar 0'dan numaralanır. Başlangıç durumunda (initial state) tahta boştur. Hoca ağaçta ilk satır için iki dal çizdi: ilk vezir (0,0) gözüne ya da (0,1) gözüne.

İlk vezir (0,0) olduğunda: İkinci vezir bir sonraki satıra konacak. (1,0) olamaz, çünkü ilk vezirin altındadır. (1,1) olamaz, çünkü çaprazındadır. Geriye (1,2) ve (1,3) kalır.

- İkinci vezir (1,2) olursa üçüncü satırda (2,0), (2,1), (2,2) ve (2,3) gözlerinin hiçbirine vezir konamaz. Ağaçta daha aşağı inmeye gerek kalmaz. Bu düğüm budanır (prune).
- İkinci vezir (1,3) olursa üçüncü satırda vezirin konabileceği tek yer (2,1)'dir. Bu durumda dördüncü satırın dört gözünün hiçbirine vezir konamaz. Bu dal da elenir.

Demek ki ilk satırda veziri ilk göze koyduğumuzda çözüm üretme şansı ortadan kalkar.

İlk vezir (0,1) olduğunda: İkinci satırın ilk üç gözüne vezir konamaz. Konabilecek tek yer (1,3)'tür. Üçüncü satırda vezir ilk sütuna, (2,0)'a konabilir; üç vezir birbirini tehdit etmez, bu bir seçenektir. Son olarak dördüncü satırda vezir üçüncü göze, (3,2)'ye konur.

İlk çözüm: (0,1), (1,3), (2,0), (3,2). Dört vezir 4×4 tahtada birbirini almayacak şekilde yerleşmiştir. Dizi gösterimiyle $[1, 3, 0, 2]$.

Hoca böylece durum uzayı ağacını oluşturarak problemi kâğıt üzerinde çözdüğünü, ağacın nasıl budandığını ve umut veren ile umut vermeyen düğümlerin neler olduğunu gösterdiğini söyledi.

Python ile kontrol edildi: Geri izleme 4-vezirde ilk çözüm olarak $[1, 3, 0, 2]$'yi bulur. İkinci çözüm bunun aynası olan $[2, 0, 3, 1]$'dir. Bütün çözümler için kök dahil 17 düğüm ziyaret edilir.

Soru: Buradaki vezirler kenarlara birlik kaydırma ve üçlük aralıklarla yerleşmiş gibi görünüyor. N-vezir problemini bunun üzerinden çözen bir formül var mı? **Cevap:** Hoca böyle bir formül bilmediğini söyledi. 4-vezirde gerçekten bir formül varmış gibi göründüğünü, ama 5, 6, 7 ve 8'de bunun denenip bir formüle oturup oturmadığına bakmak gerektiğini belirtti. (Ayrıntılı cevap "Örnek Sorular" bölümünde.)

4.4 Kaba kuvvetten farkı ne?

Aradan sonra bir öğrenci, ağaçta yine tek tek denediğimizi, kaba kuvvetten ne farkı olduğunu sordu.

Soru: Kaba kuvvet yaklaşımından ne farkı oldu, yine tek tek denemiş olmadık mı? **Cevap:** Hoca şöyle açıkladı: İki veziri yerleştirdiniz. Üçüncüyü yerleştirmeye çalıştınız ve yerleşmediğini gördünüz. O anda artık üçüncü vezirin her seçeneği için dördüncü vezire bakmanıza gerek kalmaz. Aynı şekilde ilk vezir (0,1)'e konduğu anda, ikinci satırdaki ilk üç seçenek vezirin altına ya da çaprazına geldiği için elenir. Bunlar elendiği anda altlarındaki üçüncü ve dördüncü vezir kombinasyonları da elenmiş olur. Bu yüzden yaklaşıma budamalı derinlik öncelikli arama (depth first search with pruning) da denir. İlgili noktaya kadar inersiniz; ilerisinde çözüm yoksa ilerideki bütün kombinasyonlardan kurtulursunuz.

5. Bilgisayarda N-Vezir: Çapraz Kontrol ve Dizi Gösterimi

Kâğıt üzerinde gözle baktık. Bunu bilgisayara yaptırmak için analitik bir kural vermek gerekir: Çaprazdaki iki vezirin birbirini tehdit edip etmediği nasıl hesaplanır?

5.1 Çapraz tehdit kontrolü

Hoca iki çapraz yön çizdi:

- **Birinci yön:** (0,0), (1,1), (2,2). Satır arttıkça sütun da artar.
- **İkinci yön:** (0,2), (1,1), (2,0). Satır arttıkça sütun azalır.

İki vezir arasındaki ilişkiye bakarken sütunların farkının satırların farkına eşit olup olmadığına bakılır: $c_i - c_j = r_i - r_j$ mi? Ama hangi vezirin önce geldiğine göre fark artı ya da eksi çıkar. Bu yüzden mutlak değer kullanılır. Vezir i, (r_i, c_i); vezir j, (r_j, c_j) konumunda olsun:

$$|c_i - c_j| = |r_i - r_j| \;\Rightarrow\; \text{iki vezir aynı çaprazdadır, birbirini tehdit eder.}$$

Bu kontrolle bir vezirin durum uzayı ağacında o konuma yerleşip yerleşemeyeceği bilgisayara hesaplatılır.

Soru: Yukarıdaki formülü anlamadım. **Cevap:** Hoca bir örnekle tekrar etti. Bir vezir (2,2)'de, diğeri (0,0)'da olsun. Satırları çıkarırız: $2-0=2$. Sütunları çıkarırız: $2-0=2$. Satırlar arası fark ile sütunlar arası fark eşit olduğu için bu konum bir seçenek olamaz; bu çapraz üzerindeki gözlerden birine vezir koysak iki vezir birbirini tehdit eder. Mutlak değer kullanmazsak bu kez öbür çapraz için hatalı çalışır. Örneğin (0,2) ile (2,0) karşılaştırılırken bir tarafta $0-2=-2$, diğer tarafta $2-0=2$ çıkar. İkisinin mutlak değeri 2 olduğu için bu iki vezir de birbirini tehdit eder ve konum seçenek olmaktan çıkar.

5.2 Konumları bir dizide tutmak

Hoca ikinci kritik noktanın da normalde derste "nasıl yapılır" diye sorduğu bir soru olduğunu, ama bu kez cevabı doğrudan vereceğini söyledi. Vezirlerin konumunu $n \times n$'lik bir matriste tutmak yerine tek bir satırda, yani n elemanlı bir dizide tutabiliriz. Çünkü her satırda tam bir vezir olacağını biliyoruz.

- Dizinin indisi vezirin bulunduğu satırı gösterir.
- Dizinin o gözündeki değer vezirin o satırdaki sütununu gösterir.

Hocanın örneği $[2, 3, 0, 0]$ dizisiydi: 0. satırda 2. sütunda bir vezir, 1. satırda 3. sütunda bir vezir var, diğerleri henüz yerleşmemiş. Böylece bellek karmaşıklığı (space complexity) $n \times n$'den n'e iner.

> **Ders sonrası not:** Hocanın örneğinde henüz yerleşmemiş satırlar 0 ile gösterildi. 0 da geçerli bir sütun numarası olduğu için programda bu gözleri -1 gibi geçersiz bir değerle doldurmak ya da yalnızca o ana kadar yerleştirilen satır sayısını ayrıca tutmak büyük olasılıkla daha güvenlidir. Aşağıdaki kodda ikinci yol kullanıldı.

Soru: Matris kısmını tam anlayamadım, tekrar edebilir misiniz? **Cevap:** Hoca kâğıt üzerinde vezirleri matrise yerleştirdiğimizi, ama aynı satırda birden fazla vezir olamayacağı için konumları bir dizide saklayabileceğimizi tekrarladı. Dizinin indisi vezirin satırını, gözdeki değer sütununu söyler. Böylece matrisle işlem yapmak yerine dizi kullanılır ve bellek ihtiyacı azalır.

Aynı sütun kontrolü $c_i = c_j$ ile yapılır. Her satıra tek vezir konduğu için ayrıca satır kontrolüne gerek yoktur. Bu iki kontrol birleşince bir konumun umut verici olup olmadığını söyleyen fonksiyon şöyle yazılabilir (derste kod yazılmadı, kısa notla aynı kod):

```c
#include <stdlib.h>

/* (row, col) konumuna vezir konabilir mi?
   cols[0..row-1]: önceki satırlardaki vezirlerin sütunları */
int promising(int row, int col, const int cols[]) {
    for (int i = 0; i < row; i++)
        if (cols[i] == col || abs(cols[i] - col) == row - i)
            return 0;
    return 1;
}
```

Burada $i <$ row olduğundan row $- i$ zaten pozitiftir; mutlak değer yalnızca sütun farkına uygulanır.

6. Genel Geri İzleme Algoritması

Bir öğrenci geri izlemenin karmaşıklığına değinilip değinilmeyeceğini sordu. Hoca bunun güzel bir soru olduğunu söyledi ve önce bütün geri izleme algoritmalarının mantığını temsil eden, ama hepsi için birebir aynı olmayan genel bir sözde kod (pseudocode) yazdı.

Hocanın sözlü açıklamasına göre mantık şöyledir: Durum uzayı ağacında i. noktaya geldiğinizde bir çözüm bulduysanız onu doğrudan yazdırın. Bulamadıysanız kümenize kısıtları (constraint) bozmayacak bir eleman daha ekleyin, yani yeni bir vezir yerleştirin. Yerleştirdiğiniz vezir bir çözüm oluşturuyorsa yazdırın. Oluşturmuyorsa kalan elemanlar arasından yine kısıtları bozmayan birini alıp yerleştirin. Hoca, alt küme toplamında da aynısını yaptığımızı hatırlattı: Sayıları ekledik, eklerken geldiğimiz noktanın bir çözüm olup olmadığına baktık, değilse bir sonraki noktaya ilerledik.

```
Backtrack(X[1..i]):
    if X[1..i] bir çözüm ise:
        yazdır
    else:
        for her x in S(i+1)          # kısıtları bozmayan adaylar
            X[i+1] = x
            Backtrack(X[1..i+1])
```

Hoca bir noktaya dikkat çekti: Bazen tek bir çözüm değil birden fazla çözüm vardır ve hepsinin paylaşılması istenebilir. Örneğin toplamı 9 olan iki sayıyı ilk adımda buldunuz, ama başka kombinasyon olup olmadığı soruldu. O zaman bütün ağacı oluşturmanız, umut vermeyen dalları budayarak bütün seçenekleri elemeniz gerekir.

Soru: Bu vezir problemini başka bir yerde de kullanıyor muyuz? Sudoku çözmeye çok benziyor. **Cevap:** Hoca iki şey söyledi. Birincisi, bu problem satrançta bir sonraki hamlenin ne tür bir risk taşıdığının hesaplanmasında kullanılan bir fikirdir. İkincisi, burada kısıtlar vardır. Vezir problemi bunu çok iyi anlattığı için, bir seçeneğin diğer seçeneği çürütmeyecek şekilde kullanılması gereken bütün problemlerde bu yaklaşım kullanılabilir. Hocanın örneği ders programıydı: Dersleri öyle yerleştirin ki hiçbir hocanın dersi kendisiyle çakışmasın, öğrencilerin ders almasında da bir kilitlenme (deadlock) oluşmasın. Aynı mantıktır.

> **Ders sonrası not:** Hoca burada bu tür yerleştirme problemlerinin "NP problemleri olduğunu ve polinom zamanda çözülemeyeceğini" söyledi; doğrusu bir problemin NP sınıfında olması onun polinom zamanda çözülemeyeceğini kanıtlamaz, P ile NP'nin eşit olup olmadığı hâlâ açık bir sorudur. Ayrıca N-vezirin tek bir çözümünü bulmak için polinom zamanlı açık kurgular vardır (bkz. "Örnek Sorular"). Zor olan, bütün çözümleri saymak ya da büyük olasılıkla bazı kısıtlı sürümleridir.

> **Ders sonrası not:** Hoca geri izlemenin N-vezir gibi problemleri "doğrusal zamanda (linear time) çözme imkânı sağladığını" söyledi; doğrusu geri izleme yalnızca kapsamlı aramaya göre denenen düğüm sayısını azaltır, en kötü durumda yine üstel çalışabilir. Hocanın kendisi de birkaç dakika sonra karmaşıklığın girdiye bağlı olduğunu ve genel olarak kesin ifade edilemeyeceğini anlattı (Bölüm 8). Python ile kontrol: Bütün çözümler için ziyaret edilen düğüm sayısı $n = 4, 5, 6, 7, 8$ için sırasıyla 17, 54, 153, 552, 2057'dir; bu büyüme doğrusal değildir.

7. Arama Uzayını Küçültme Hileleri

Hoca, problemleri çözerken bazılarının fark ettiği, bazılarının fark etmediği iki ek hamle yaptığımızı söyledi.

Simetri (ayna): N-vezirde veziri soldan başlayarak yerleştirdik. Sağdan başlayıp yerleştirseydik, tahtanın ortasından bir çizgi çektiğimizde iki tarafın birbirinin aynası olduğunu görürdük. Bir tarafta veziri bir göze, diğer veziri başka bir göze koymakla bunların aynadaki karşılıklarına koymak aynı anlama gelir. Bu yüzden ilk satırda belli bir yerden sonraki kısma bakmaya gerek yoktur, çünkü onun aynasını daha önceki turlarda zaten kontrol etmiş oluruz. Bu yansıtma (mirror) yöntemiyle seçeneklerin bir kısmı elenebilir.

Sıralama: Alt küme toplamında karışık verilen sayıları sıraladık. Sıralama sayesinde, hedefe ulaşmak için önümüzde yeterince sayı ya da değer kalmadığında o kombinasyonları kontrol

etmemize gerek kalmaz. Bu, geri izlemenin avantajını büyütür ve karşılaştırma sayısını azaltır.

Hocaya göre geri izleme problemlerinde bu tür küçük hamlelerle arama uzayı daha da küçültülebilir.

8. Geri İzlemenin Karmaşıklığı

Hoca karmaşıklık için net bir cevap olmadığını söyledi. Girdilerin ve kombinasyonların veriliş biçimi, karmaşıklığın şudur dememize izin vermez. Bunun belirli bir standart sapması vardır ve bu sapma yeri geldiğinde yüksek olabilir.

Hocanın kaba hesabı şöyleydi: İlk x_1 değeri için c_1 seçenek olduğunu, her birinin seçilme olasılığının $1/c_1$ olduğunu varsayalım. Kökün çocuklarına c_1 farklı şekilde ilerlenir. İkinci seviyede c_2 devreye girer ve ikinci seviyedeki çocuklara ulaşmak için $c_1 \cdot c_2$ düğüm oluşur. Aşağı doğru dallandıkça düğüm sayısı bu oranda artar:

$$1 + c_1 + c_1c_2 + c_1c_2c_3 + ...$$

Fakat ağacın hangi noktada budanacağını ve i. düğümün çocuklarına bakıldığında hangi olasılıkların ortaya çıkacağını bilmediğimiz için net bir şey söylemek mümkün değildir. Sonuç problemin kendisine ve verinin dağılımına bağlıdır. Hoca, vezir problemi, alt küme toplamı ve Hamilton (Hamiltonian) problemi için geri izlemenin oldukça iyi sonuç verdiğini, ama sayılar büyüdüğünde ve verilen sayıların kombinasyonu değiştiğinde problemi düşündüğünüz kadar hızlı çözemeyebileceğinizi söyledi. Literatürde de karmaşıklığın, girdiler değiştikçe sapma çok yüksek olduğu için genelleştirilemediği görülür.

Hoca son olarak şunu paylaştı: Geri izlemede yapılan budama, ağaç taramasının zaman ve bellek açısından bir kaynak sınırına takılmadan sonlanmasını ümit ederek yapılır. Yola çıktığınızda sonucun polinom zamanda gelmesini ve belleğin taşmadan, bir bölütleme hatası (segmentation fault) almadan bitmesini beklersiniz. Derste çok küçük sayılarla çalışıldı. Sayıları büyüttüğünüzde muhtemelen sonuç alırsınız, ama bazı durumlarda istenen zamanda ve bellekle sonuçlanmama ihtimali vardır. Kapsamlı aramaya göre olasılık sayısını çok düşürdüğünüz için budamanın lehinize sonuçlanmasını beklersiniz, ama bu her zaman geçerli bir sorun çözücü değildir.

9. Dal ve Sınır: Optimizasyon Problemleri

Dal ve sınır (branch and bound) optimizasyon problemlerini çözmek için kullanılan bir yaklaşımdır. Bir optimizasyon probleminin hedefi bir şeyi ya en büyük (maximize) ya da en küçük (minimize) yapmaktır. Bunu çoğu zaman belirli kısıtlar altında yapmak gerekir. Bir amaç fonksiyonu (objective function) kurulur ve en iyi sonuç aranır.

Optimizasyon problemlerinde iki tür çözüm vardır:

- **Uygun çözüm (feasible solution):** Arama uzayında bütün kısıtlarınızı karşılayan çözümdür. Bunların hepsi makul çözüm sayılır.
- **En iyi çözüm (optimal solution):** Uygun çözümler arasında amaç fonksiyonuna göre en iyi sonucu verendir. Aranan budur.

Daha önce bir öğrenci geri izlemenin, doğal dil işlemede kullanılan ışın aramasına (beam search) benzeyip benzemediğini sormuştu. Öğrencinin açıklamasına göre orada cümle sonu işareti (end of sentence token) görülene kadar ilerleniyor ve en yüksek olasılıklı cümle aranıyordu. Hoca bir benzerlik olup olmadığını bilmediğini, yanlış bilgi vermek istemediğini söyledi. Orada bir en büyükleme problemi olduğu öğrenilince, dal ve sınırın geri izlemenin optimizasyon problemlerine uyarlanmış hali gibi düşünülebileceğini ve öğrencinin bahsettiği algoritmanın muhtemelen ona benzediğini belirtti.

> **Ek bilgi (derste anlatılmadı):** Işın araması da bir ağaç araması yapar, ama her seviyede yalnızca en iyi k düğümü (ışın genişliği) tutar, diğerlerini sınır hesabı yapmadan atar. Bu yüzden hızlıdır, ama en iyi çözümü bulacağını garanti etmez. Dal ve sınır ise yalnızca, sınırı o ana kadarki en iyi çözümden daha iyi olamayacağını gösteren dalları budar. Bu yüzden en iyi çözümü garanti eder, ama en kötü durumda üstel sayıda düğüme bakabilir.

10. Atama Problemi ve En İyi Öncelikli Dal ve Sınır

10.1 Problem

Hoca günlük hayatta da sık karşılaşılan atama problemini (assignment problem) ele aldı: n kişi ve n iş vardır. Problem $n \times n$'lik bir matrisle gösterilir. Her kişiye bir iş atanacak ve toplam maliyet en az olacaktır. Örnekte 4 kişi (A, B, C, D) ve 4 iş vardı:

	İş 1	İş 2	İş 3	İş 4
A	9	2	7	8
B	6	4	3	7
C	5	8	1	8
D	7	6	9	4

Hoca örneğin A kişisi birinci işi yaparsa 9, ikinci işi yaparsa 2 birim (saatlik ücret gibi) alacağını söyledi. Bütün kişilerin bütün işleri yapabilecek yeteneği ve kapasitesi aynıdır. "A birinci işi daha iyi yapar" gibi bir kısıt yoktur; yalnızca maliyetler farklıdır. İşveren hangi işe kimi atayacağını en az maliyetle belirlemek ister. Amaç bütün kombinasyonları denemeden, mümkün olan en az adımda sonuca ulaşmaktır.

10.2 Kök: ilk alt sınır

İlk akla gelecek çözüm, her satırdaki kişinin en ucuz işini seçmektir: A için 2, B için 3, C için 1, D için 4. Toplam:

$$2 + 3 + 1 + 4 = 10$$

Sıkıntı şudur: Bu kombinasyonda üçüncü işe iki kişi (B ve C) atandı, birinci işe hiç atama yapılmadı. Bu istediğimiz bir durum değildir. Ama bu değerler farklı sütunlara denk gelseydi, elde edebileceğimiz en iyi kombinasyon bu olurdu. Bu yüzden 10, problemin bir alt sınırıdır (lower bound): Hiçbir geçerli atama 10'dan daha ucuz olamaz. Bir en küçükleme problemiyle uğraştığımız için alt sınır kullanırız. Bu değer başlangıç (start) düğümündeki olabilecek en düşük maliyettir.

Yine bir durum uzayı ağacı kurulur. Her adımda seçenekler değerlendirilir ve bir sonraki adımda nereye gidileceğine karar verilir.

10.3 Birinci seviye: A'nın işi

Kökten dört seçenek çıkar: A'ya 1., 2., 3. ya da 4. iş verilebilir. Her seçenek için alt sınır şöyle hesaplanır: A'nın seçilen işinin maliyeti alınır. Diğer kişiler için, A'ya verilen sütun hariç, her satırın en küçük değeri toplanır. Diğer kişilerin kendi aralarında çakışması bu adımda önemsenmez.

- **A → 1:** 9 kesin. Geri kalanların en küçükleri yine $3 + 1 + 4$. Hoca, 3 ile 1'in yine aynı işe düştüğünü, ama bu adımda bununla ilgilenmediğimizi, yalnızca A'yı 1'e atamakla ilgilendiğimizi vurguladı. Alt sınır $9 + 3 + 1 + 4 = 17$. Yani A'ya birinci işi verirsek, diğer işleri dağıttığımızda ulaşabileceğimiz en iyi değer 17'dir.
- **A → 2:** $2 + 3 + 1 + 4 = 10$.
- **A → 3:** A 3. işi aldığı için B ve C artık 3. işi kullanamaz. B'nin 3 dışındaki en iyi seçeneği 4, C'nin 5, D'nin hâlâ 4. Alt sınır $7 + 4 + 5 + 4 = 20$.
- **A → 4:** D artık 4'ü alamaz, satırındaki 4 dışındaki en küçük değer 6. Alt sınır $8+3+1+6 = 18$.

Seçenekler arasındaki en iyisi 10'dur ve doğal olarak onun peşinden gidilir. Hoca, her adımda o anki en iyi sınıra sahip düğümden devam etme stratejisine **en iyi öncelikli dal ve sınır (best-first branch and bound)** dendiğini söyledi.

10.4 İkinci seviye: A → 2 altında B'nin işi

A'ya 2. iş verildi. B'ye en düşükten başlayarak kalan işler verilir. B'ye 2. iş verilemez.

- **B → 1:** $2 + 6 = 8$. Geriye C için 1 (iş 3), D için 4 (iş 4) kalır. Alt sınır $2 + 6 + 1 + 4 = 13$.
- **B → 3:** $2 + 3 = 5$. C artık 3. işi, yani 1'i alamaz; 5'i alır, toplam 10. D'ye 4 verilir. Alt sınır $2 + 3 + 5 + 4 = 14$.
- **B → 4:** $2 + 7 = 9$. C'ye 1 eklenir, toplam 10. D 2. ve 4. işi alamaz; alabileceği en küçük değer 7'dir. Alt sınır $2 + 7 + 1 + 7 = 17$.

Bu noktada elimizde 17, 20, 18 (birinci seviyeden) ile 13, 14, 17 (ikinci seviyeden) var. En küçüğü 13 olduğu için yola onunla devam edilir. Diğerlerine şimdilik gidilmez.

10.5 Üçüncü seviye ve sonuç

A → 2, B → 1 olduğuna göre C'nin iki seçeneği vardır: 3 ya da 4. D de kalan işi alır.

- **C → 3, D → 4:** $2 + 6 + 1 + 4 = 13$. Bütün işler atanmıştır ve maliyet gerçekten 13'tür.
- **C → 4, D → 3:** $2 + 6 + 8 + 9 = 25$.

Bütün işlerin atandığı bir yaprakta 13 maliyetine ulaşıldı. Açıkta kalan bütün düğümlerin alt sınırları (14, 17, 17, 18, 20) 13'ten büyük olduğu için bu dalların hiçbiri 13'ten iyi bir atama veremez ve hepsi budanır.

Sonuç: A → 2, B → 1, C → 3, D → 4. En az toplam maliyet 13.

Hoca yolculuğu şöyle özetledi: Yola 10 alt sınırıyla çıktık. Ama 10 kısıtlara uymadığı için ilerleyen adımlarda bu değer 13, 14 ve 17'ye dönüştü. İlk kararı verirken 17, 10, 20 ve 18 arasından 10'u seçmiştik. Eğer başka bir seçenekte de aynı değer çıksaydı, ağacın diğer bir bölgesindeki bir seçeneğin en iyi olma ihtimali olurdu ve oraya da bakmak gerekirdi. Bu örnekte böyle bir durum olmadığı için bunu tartışmaya gerek kalmadı.

Python ile kontrol edildi: Aynı alt sınır kuralıyla kök 10, birinci seviye $[17, 10, 20, 18]$, ikinci seviye $[13, 14, 17]$, yapraklar 13 ve 25 çıktı. Bütün $4! = 24$ atama denendiğinde en küçük maliyet 13'tür ve atama aynıdır (A → 2, B → 1, C → 3, D → 4). İkinci en iyi atama 14'tür.

> **Ders sonrası not:** Hoca C → 4, D → 3 yaprağı için 25'i bulduktan sonra bunun "geçerli olmayan bir sonuç" olduğunu söyledi; doğrusu bu atama da geçerli (uygun) bir atamadır, yalnızca 13'ten pahalı olduğu için en iyi değildir.

Soru: B → 1'de alt sınırın 13 olduğu belli olmuyor mu? C3-D4 ya da C4-D3 kombinasyonlarını çizmeden bir önceki adımda 13 olduğu belli değil mi? **Cevap:** Hoca belli olmadığını söyledi. 13'e C3-D4'ün mü yoksa C4-D3'ün mü ulaşacağını bilmeyiz. İkisinin de 13 olmama olasılığı vardır, 14 de olabilirdi. Bu örnekte denk geldi ve 13 çıktı. Tıpkı A → 2'de alt sınırın 10 olması gibi: 10 çıkma ihtimali vardı ama çıkmadı. Bu yüzden iki kombinasyonu da kontrol etmek gerekir.

> **Ek bilgi (derste anlatılmadı):** Bu sorunun kısmen haklı olduğu bir yan vardır. B → 1'de, kalan iki kişinin satır en küçükleri (C için iş 3, D için iş 4) farklı sütunlara düştüğü için, bu sınır büyük olasılıkla gerçek bir atamaya karşılık gelir; satır en küçükleri çakışmadığında alt sınır gerçek maliyete eşittir. Ama algoritma bunu genel olarak bilmez; yaprak düzeyine inip atamayı tamamlamadan bir değer "çözüm" sayılmaz.

Hoca, haftaya dal ve sınırla ilgili bir örnek daha çözüp konuyu kapatacağını söyledi.

Örnek Sorular

Soru 1 (derste sözlü, 00:40:15): 1×1, 2×2 ve 3×3 tahtalarda N-vezir probleminin çözümü var mıdır? En az kaçlık bir tahta gerekir?

Çözüm (derste verildi): Hoca 2×2 ve 3×3 tahtalarda vezirlerin birbirini yemeyecek şekilde yerleştirilemeyeceğini, en az 4×4'lük bir tahta gerektiğini anlattı. Hoca 1×1 için de çözüm yok dedi.

> **Ders sonrası not:** $n = 1$ için tek vezir tek kareye konur ve bu bir çözümdür. Python ile sayılan çözüm sayıları: $n = 1$: 1, $n = 2$: 0, $n = 3$: 0, $n = 4$: 2, $n = 5$: 10, $n = 6$: 4, $n = 7$: 40, $n = 8$: 92.

Soru 2 (derste sözlü, 00:48:08): 4-vezir çözümündeki vezirler düzenli aralıklarla yerleşmiş görünüyor. N-vezir problemini çözen bir formül var mı?

Hoca böyle bir formül bilmediğini, 5, 6, 7 ve 8 için denenmesi gerektiğini söyledi.

> **Çözüm (derste verilmedi):** Tek bir çözüm üreten açık kurgular vardır. Satır ve sütunları 0'dan sayalım. n mod 6 değeri 2 ya da 3 değilse şu dizi her zaman bir çözümdür: önce tek sütunlar $1, 3, 5, ...$, sonra çift sütunlar $0, 2, 4, ...$ $n = 4$ için bu $[1, 3, 0, 2]$'dir, yani derste bulunan çözümün ta kendisidir. $n = 6$ için $[1, 3, 5, 0, 2, 4]$ elde edilir. Python ile $4 \leq n < 40$ aralığında n mod $6 \notin \{2, 3\}$ olan bütün n değerleri için bu dizinin geçerli olduğu kontrol edildi. n mod $6 \in \{2, 3\}$ olduğunda ise bu basit dizi çalışmaz (aynı aralıkta hiçbirinde geçerli değil); bu durumlar için büyük olasılıkla dizinin bazı elemanlarının yer değiştirildiği farklı bir kurgu kullanılır. Yani tek bir çözüm bulmak kolaydır; bütün çözümleri bulmak ya da saymak için geri izleme gibi arama yöntemleri gerekir.

Soru 3 (derste sözlü, 00:59:36): 4-vezir ağacında yine tek tek denemiş olmadık mı? Kaba kuvvetten farkı ne?

Çözüm (derste verildi): Üçüncü vezirin yerleşemediği görüldüğü anda üçüncü vezirin her seçeneği için dördüncü vezire bakmaya gerek kalmaz. İlk vezir (0,1)'e konunca ikinci satırdaki üç seçenek elenir ve altlarındaki bütün üçüncü ve dördüncü vezir kombinasyonları da elenmiş olur.

Soru 4 (derste sözlü, 01:04:21): Vezirlerin konumunu $n \times n$'lik bir matris tutmadan nasıl saklarız?

Çözüm (derste verildi): n elemanlı bir dizide. İndis satırı, değer sütunu gösterir. Her satırda tek vezir olduğu için bu yeterlidir. Bellek n^2'den n'e iner.

Soru 5 (derste sözlü, 01:07:41): (2,2) ve (0,0)'daki iki vezir birbirini tehdit eder mi? Peki (0,2) ve (2,0)?

Çözüm (derste verildi): İlk durumda satır farkı $2 - 0 = 2$, sütun farkı $2 - 0 = 2$; eşit oldukları için tehdit vardır. İkinci durumda sütun farkı $0 - 2 = -2$, satır farkı $2 - 0 = 2$ çıkar; mutlak değerleri eşit olduğu için yine tehdit vardır. Mutlak değer kullanılmazsa ikinci durum gözden kaçar.

Soru 6 (derste sözlü, 01:41:30): Atama probleminde B → 1 düğümünün alt sınırı 13 bulununca, C ve D'nin iki kombinasyonunu çizmeden sonucun 13 olduğu belli olmaz mı?

Çözüm (derste verildi): Olmaz. Alt sınır yalnızca bir sınırdır, gerçekleşmeyebilir. 13'e hangi kombinasyonun ulaşacağı da bilinmez. İki kombinasyon da kontrol edilir: C → 3, D → 4 için 13; C → 4, D → 3 için 25.

Dersin Toparlaması

Bu derste geri izleme, budamalı derinlik öncelikli arama olarak tanıtıldı. Durum uzayı ağacında her düğüm kısmi bir çözümdür. Umut vermeyen bir düğüme gelindiğinde onun altındaki bütün kombinasyonlar tek seferde elenir. Alt küme toplamında sayıları sıralamanın "toplam çok büyük" ve "toplam çok küçük" budamalarını mümkün kıldığı görüldü. N-vezirde 4-vezir ağacı elle kuruldu, ilk çözüm $[1, 3, 0, 2]$ bulundu. Bilgisayarda çapraz kontrolün $|c_i - c_j| = |r_i - r_j|$ ile yapıldığı ve konumların n elemanlı bir dizide tutulabileceği anlatıldı. Simetri ve sıralama gibi küçük hamlelerin arama uzayını daha da küçülttüğü, ama geri izlemenin karmaşıklığının girdiye bağlı olduğu ve genel olarak kesin ifade edilemeyeceği vurgulandı.

Dersin ikinci kısmında aynı ağaç fikri optimizasyon problemlerine taşındı. Dal ve sınırda her düğüm için ulaşılabilecek en iyi değere bir sınır hesaplanır. En iyi öncelikli stratejide her adımda en iyi sınıra sahip düğümden devam edilir. Atama probleminde satır en küçüklerinden gelen alt sınırla başlanıp en iyi atama (A → 2, B → 1, C → 3, D → 4, maliyet 13) bulundu. Hoca haftaya dal ve sınırla ilgili bir örnek daha çözerek konuyu kapatacağını söyledi.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.